Lab 2A: The Agentic Build Loop

Duration: 30-45 min

After: Installation & Agentic Loop slides

Goal

Today you're not learning how to prompt an AI. **You're learning how to manage an AI engineer.**

In traditional development, you would build this application file-by-file, dependency-by-dependency. Today you will describe the system and let an AI engineer build it — then inspect, critique, and extend what it produced.

Your task is not to judge whether Claude writes perfect code. Your task is to evaluate whether it can move from idea → running system faster than traditional development.

---

1. Pre-Flight Check

Open a terminal and verify your environment:

> ***Which terminal?*** *Use whatever you normally use — PowerShell, Windows Terminal, Mac Terminal, or Linux shell. Claude Code works in all of them.*

```
node -v
```

Expected: `v18.x.x` or higher

```
git --version
```

Expected: `git version 2.x.x`

```
claude --version
```

Expected: A version number like `1.x.x`

> ***Not installed?*** *See the Installation slide or ask your instructor.*

□Pre-Flight Complete

- [ ] Node.js 18+ installed
- [ ] Git installed
- [ ] Claude Code installed

---

2. What You Should See

During the build, Claude will create files, install packages, and run commands — all autonomously. It will ask permission before executing shell commands. Approve these or select "Yes, and don't ask again" to reduce prompts.

If Claude hits an error, it will read the output, diagnose the problem, and fix it without your help.

⚠ **The terminal may pause for 10-30 seconds while Claude reasons.** That is normal — do not interrupt it.

---

3. Create Your Project and Start Claude

```
mkdir business-dashboard
cd business-dashboard
claude
```

You're now in an interactive Claude Code session. **From this point forward, everything happens inside Claude.**

---

4. The Build Prompt

Copy and paste this prompt exactly:

```
Initialize this as a git repo and build me a business dashboard web application:

Tech stack: Node.js, Express, SQLite (via better-sqlite3), vanilla HTML/CSS/JS frontend
No external CSS frameworks — write clean, modern CSS

Database tables:
1. metrics — id, name, value (number), category, recorded_at
2. tasks — id, title, status (pending/in-progress/completed), priority (high/medium/
low), assigned_to, created_at, completed_at

API endpoints:
- GET /api/metrics (filter by ?category=)
- POST /api/metrics
- GET /api/tasks (filter by ?status= and ?priority=)
- POST /api/tasks
- PUT /api/tasks/:id
- DELETE /api/tasks/:id
- GET /api/dashboard (summary: all metrics + task counts by status + high priority task
s)

Frontend: Dark theme dashboard at / showing metrics as responsive cards
and tasks as an interactive list with status badges and priority indicators.
Seed the database with 8 sample metrics across categories (financial, customers,
operations, hr) and 5 sample tasks.

Initialize npm, install dependencies, create everything, and start the server.
Use port 3100 (or PORT env var).
Include a .gitignore for node_modules and *.db files.
```

Watch the agentic loop in action:

Phase	What Claude Does
Plan	Analyzes requirements, decides on file structure
Execute	Creates files, runs `npm install`, writes code
Validate	Checks for errors, tests the server

> ⚠ ***Do not type while Claude is working.*** *Wait until it finishes the build loop and returns to the prompt. Typing mid-execution can break the session.*

The self-healing loop: If Claude hits an error — a missing dependency, a port conflict, a syntax mistake — it reads the error, diagnoses the cause, fixes it, and re-runs. No hand-holding required.

---

5. See It in the Browser

Open your browser to **http://localhost:3100**

You should see a dark-themed dashboard with metric cards and tasks.

Reality Check

This example is intentionally small so you can see the entire system. In real projects, Claude Code works best when the repo already exists, architecture patterns are established, tests exist, and CLAUDE.md defines conventions. In those environments Claude acts less like a code generator and more like a junior-to-mid engineer who can read the entire codebase instantly.

---

6. Test the System Through Claude

```
Test the API: add a new metric called "New Leads This Week" with value 42
in the "sales" category. Then add a task "Review Q2 projections" with
high priority assigned to me. Then complete task 1.
Show me the results after each operation.
```

Refresh the dashboard in your browser. The new data should appear.

---

7. Your First Git Commit

```
Commit everything with the message "feat: initial business dashboard with API and UI"
```

> ***You may see a Skills prompt.*** *If Claude asks to use a "commit" skill, select* ***Yes*** *— this is normal. You'll learn more about Skills in Day 3.*

---

8. Reverse Engineer the System

Now make Claude explain what it built — and why.

```
Explain the architectural decisions you made while building this system.
```

Claude will walk through its choices: Express routing, SQLite, frontend separation, API design. This shows **reasoning**, not just generation.

Then challenge it:

```
Identify the 3 biggest architectural weaknesses in this project.
Rank them by risk if this system went to production.
What would break first under scale?
```

This isn't code generation — this is an architectural review.

---

9. Vague vs. Specific Prompting

Try this vague prompt first:

```
Add validation.
```

Claude may enter plan mode and present options. **Choose option 3 (manually approve edits)** — do not clear context mid-lab.

Look at what Claude does — it guesses which routes, which fields, what error format. Then ask:

```
Why did you choose that validation approach?
```

Now try the specific version:

```
In @src/index.js, modify only the POST /api/tasks route. Validate:
- title: must be non-empty string, max 200 characters
- priority: must be one of "high", "medium", "low"
- assigned_to: optional, but if provided must be non-empty string
Return 400 with { error: "description of what's wrong" } for invalid requests.
```

Then ask:

```
Which instructions in my prompt changed your implementation?
```

Compare the outputs. **The specific prompt produces exactly what you want on the first try.** Notice how prompt constraints alter architectural decisions.

> ***Specificity isn't extra work — it's less work.*** *One precise prompt beats three vague ones.*

---

10. Quick Reflection

Which part surprised you most?

- □ Claude writing the entire project structure
- □ Claude fixing its own errors
- □ The working UI from a text description
- □ The difference between vague and specific prompts

If Something Goes Wrong

Problem	Solution
Claude stops mid-build	Type `exit`, then `claude` again. Tell Claude: "The build stopped before finishing. Continue building the dashboard application."
Port 3100 in use	Tell Claude: "Port 3100 is in use. Switch to port 3456."
`node -v` not found	Install Node.js from nodejs.org (LTS version)
`claude` not found	Run: `curl -fsSL https://claude.ai/install.sh | bash` (Mac/Linux) or see Installation slide (Windows)
Permission errors on npm	Tell Claude: "I'm getting permission errors on npm install. Fix it."
`tmpclaude*` files appear	Ignore — Claude cleans these up automatically

Go Deeper

Controlled Refactor

```
Refactor the backend so all database access is isolated into a db module
instead of being mixed with routes. Explain what changed and why this
structure is better.
```

Watch Claude restructure the codebase while maintaining functionality.

Add a Feature

```
Add a chart to the dashboard showing metrics grouped by category
using Chart.js. Install the dependency and update the frontend.
```

You just added a visual feature to a running system with one prompt.

□Lab 2A Checkpoint

- [ ] A running Express server on port 3100
- [ ] SQLite database with metrics and tasks
- [ ] Working web UI at http://localhost:3100
- [ ] Input validation on POST /api/tasks
- [ ] First git commit

> ***Keep Claude running. Return to slides when your instructor is ready.***